Ghid deploy VM · companion MISTRAL (test)

Branch `MISTRAL-experimenting-rai-documents` · companion **v1.13.1 / build 73**
Folder nou pe VM · Docker DB · Cloudflare tunnel · Vercel (opțional) · 1 octombrie 2026

VM acasă | Docker DB | Cloudflare | Vercel | Mistral OCR

Ce obții: o instanță izolată (folder + DB + porturi) a companionului RAI Documente, pe branch-ul MISTRAL, gata de login, upload aviz și OCR cloud — fără să strici Transitix-ul / companion-ul deja rulat pe aceeași mașină.

Important: pe același VM poți avea deja Postgres pe `:5435` și API pe `:3002`. Ghidul folosește porturi **paralele** (`:5436` / `:3003`) și un volume Docker separat. Dacă porturile sunt libere, poți lăsa 5435 / 3002 — schimbă doar consistent peste tot.

0. Ce ai nevoie pe VM

- Linux (Ubuntu 22.04+ e ok) cu acces SSH
- **Docker** + plugin `compose` (`docker compose version`)
- **Node.js 22** + npm (`node -v`)
- **Git**
- Cheie **Mistral** (`MISTRAL_API_KEY`) pentru OCR pe poze/scanuri
- **cloudflared** (deja instalat pe VM-ul tău) — pentru HTTPS pe domeniu
- Opțional: cont **Vercel** dacă vrei UI pe Vercel + API pe tunnel

```
sudo apt update
# Node 22 (exemplu NodeSource) + Docker conform documentației Docker pentru distro-ul tău
node -v && npm -v && docker compose version && git --version
cloudflared --version
```

1. Folder nou + clone pe branch

Alege un path care *nu* e folderul Transitix deja în producție / demo:

```
mkdir -p ~/apps
cd ~/apps
git clone https://github.com/Georgiana-Ghimes/TransitiX.git transitix-mistral
cd transitix-mistral
git fetch origin
git checkout MISTRAL-experimenting-rai-documents
git pull origin MISTRAL-experimenting-rai-documents
git log -1 --oneline
# aștepți ceva de genul: · v1.13.1 / build 73 ·
```

Repo privat? Folosește SSH (`git@github.com:…`) sau un personal access token la HTTPS.

2. Postgres izolat în Docker

Fișierul `docker-compose.mistral-test.yml` e **deja în repo** (rădăcină). Nu îl creezi pe mână — după `git pull` doar îl pornești. Nu refolosi volumul companion-ului vechi.

```
cd ~/…/MISTRAL-experimenting-rai-documents   # folderul clone-ului
git pull origin MISTRAL-experimenting-rai-documents
ls -la docker-compose.mistral-test.yml

# Opțional: schimbă parola în YAML înainte de primul up
# (trebuie să coincidă cu DATABASE_URL din server/.env)
nano docker-compose.mistral-test.yml

docker compose -f docker-compose.mistral-test.yml up -d
docker compose -f docker-compose.mistral-test.yml ps
# health: healthy · port host 5436
```

Default în fișier: DB `transitix_rai_mistral`, user `postgres`, parolă `MistralDb` — aliniază `DATABASE_URL` la fel.

3. Fișiere env — ce schimbi

Pe un folder dedicat e cel mai simplu: totul în `server/.env` (fără dansul `COMPANION=1` / `.env.companion`). UI-ul companion se bake-uiește la `npm run build:companion` din modul Vite, nu din acest env.

```
cp server/companion.env.example server/.env
nano server/.env   # sau vim / code
```

3.1 Valori obligatorii în `server/.env`

Cheie	Valoare pentru test pe VM	De ce
`PORT`	`3003`	API + UI pe un singur origin; evită conflict cu :3002
`DATABASE_URL`	`postgresql://postgres:MistralDb@127.0.0.1:5436/transitix_rai_mistral`	Același user/parolă/DB/port ca în compose
`JWT_SECRET`	string lung aleator (*diferit* de celelalte instanțe)	Tokenii nu trebuie să fie valabili pe alt companion
`JWT_EXPIRES_IN`	`24h`	Ok pentru test
`JWT_REFRESH_EXPIRES_IN`	`7d`	Ok pentru test
`UPLOAD_DIR`	`uploads`	Relativ la `server/`; rămâne pe disc pe VM
`CLIENT_ORIGIN`	vezi mai jos	CORS + link-uri din email; trebuie să fie URL-ul cu care deschizi app-ul
`OCR_PROVIDER`	`mistral`	Forțează cloud OCR
`MISTRAL_API_KEY`	cheia ta reală	Fără ea: health `none` / pozele nu se citesc

3.2 `CLIENT_ORIGIN` — alege una

Cum testezi	Exemplu `CLIENT_ORIGIN`	`TRUST_PROXY`
Din LAN, pe IP:port	`http://192.168.x.x:3003`	nu seta (sau `0`)
Doar pe VM (localhost)	`http://127.0.0.1:3003`	nu seta
HTTPS prin Cloudflare tunnel	`https://documente-mistral.domeniul-tau.ro`	`TRUST_PROXY=1`
UI pe Vercel + API pe tunnel	`https://….vercel.app` (+ opțional și hostname-ul tunnel)	`TRUST_PROXY=1`

Dacă deschizi app-ul pe un host/port pe care `CLIENT_ORIGIN` nu îl listează, login / cookie / reset pot eșua. Poți pune mai multe origini separate prin virgulă dacă codul din proiect le acceptă pe listă — pentru test, ține *una*, aceeași pe care o tastezi în browser.

3.3 Opționale (utile la test)

```
# Email (fără cheie = stub: link-ul de reset apare în log / răspuns API)
RESEND_API_KEY=
EMAIL_FROM=RAI Test <onboarding@resend.dev>

# Conturi noi din UI
ALLOW_SIGNUP=true

# Google login (opțional). Dacă e gol, butonul e ascuns / 503 pe /auth/google
GOOGLE_CLIENT_ID=

# OCR tuning (default-urile din exemplu sunt ok)
# MISTRAL_OCR_MODEL=mistral-ocr-latest
# OCR_TIMEOUT_MS=300000
# OCR_INTERACTIVE_TIMEOUT_MS=120000
# OCR_INTERACTIVE_MAX_PAGES=3
# OCR_CONCURRENCY=2

# Companion pe VM nu are nevoie de OSRM / VROOM / Photon pentru fluxul avize
OSRM_URL=
VROOM_URL=
PHOTON_URL=
```

3.4 Exemplu minimal complet

```
PORT=3003
DATABASE_URL=postgresql://postgres:MistralDb@127.0.0.1:5436/transitix_rai_mistral
JWT_SECRET=inlocuieste-cu-un-secret-lung-aleator-doar-pentru-mistral
JWT_EXPIRES_IN=24h
JWT_REFRESH_EXPIRES_IN=7d
UPLOAD_DIR=uploads
CLIENT_ORIGIN=http://192.168.1.50:3003
OCR_PROVIDER=mistral
MISTRAL_API_KEY=sk-...cheia-ta...
ALLOW_SIGNUP=true
RESEND_API_KEY=
EMAIL_FROM=
GOOGLE_CLIENT_ID=
ETRANSPORT_MODE=stub
```

4. Instalare dependențe + migrare + seed

```
cd ~/apps/transitix-mistral
npm ci
npm ci --prefix server

# Schema pe DB-ul nou
npm run db:migrate

# Conturi demo (doar pe mediu de test; pe production cere SEED_ALLOW_PRODUCTION=1)
NODE_ENV=development npm run db:seed --prefix server
```

Conturi după seed:
· Admin: `admin@transitix.ro` / `admin123`
· Șofer: `sofer@transitix.ro` / `sofer123`
Schimbă parolele dacă expui instanța în afara LAN-ului.

5. Build UI companion + pornire

`build:companion` bake-uiește profilul `documents`, titlul RAI și **v1.13.1 / build 73** din `package.json → companion`. API-ul servește apoi `dist/` pe același `PORT`.

```
# Build frontend companion (o dată, sau după fiecare git pull cu schimbări UI)
npm run build:companion

# Pornire API (rulează și migrate la start)
npm start --prefix server

# Log așteptat:
# Transitix API listening on http://localhost:3003
```

Verificări rapide:

```
curl -s http://127.0.0.1:3003/api/health | jq .
# ocr: "mistral"  (sau "mistral-down" dacă cheia e greșită / rețeaua blochează)
# Nu vrei "none" dacă testezi OCR pe poze.

curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:3003/
# 200 = SPA companion
```

Din LAN: deschide `http://IP_VM:3003` (același host ca în `CLIENT_ORIGIN`). Firewall: `sudo ufw allow 3003/tcp` doar dacă testezi din rețea; pe tunnel Cloudflare *nu* e nevoie de port inbound.

6. Pornire persistentă (systemd)

```
sudo tee /etc/systemd/system/transitix-mistral.service > /dev/null <<'EOF'
[Unit]
Description=Transitix companion MISTRAL (test)
After=network.target docker.service
Requires=docker.service

[Service]
Type=simple
User=YOUR_USER
WorkingDirectory=/home/YOUR_USER/apps/transitix-mistral/server
Environment=NODE_ENV=production
# .env e citit de loadEnv din WorkingDirectory
ExecStart=/usr/bin/node src/index.js
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF

# Înlocuiește YOUR_USER, apoi:
sudo systemctl daemon-reload
sudo systemctl enable --now transitix-mistral
sudo systemctl status transitix-mistral --no-pager
```

După `git pull` + schimbări UI: rulezi din nou `npm run build:companion` din root-ul proiectului, apoi `sudo systemctl restart transitix-mistral`.

7. Cloudflare Tunnel (HTTPS) — ai deja cloudflared pe VM

Recomandat pentru test: un hostname nou pe *același* tunnel pe care îl folosești deja pentru companion-ul vechi. Cloudflare dă certificatul; pe VM rămâne `http://127.0.0.1:3003`. Nu deschizi port inbound.

7.1 Vezi ce rulează acum

```
sudo systemctl status cloudflared --no-pager
# sau, dacă e rulat din user:
systemctl --user status cloudflared --no-pager

# Config tipic:
ls -la ~/.cloudflared/ /etc/cloudflared/ 2>/dev/null
sudo cat /etc/cloudflared/config.yml
# sau: cat ~/.cloudflared/config.yml

cloudflared tunnel list
```

Notează `tunnel:` (nume sau UUID) și hostname-ul existent (ex. `documente.domeniul-tau.ro` → `http://127.0.0.1:3002`).

7.2 Adaugă DNS pentru instanța MISTRAL

Pe același tunnel (înlocuiește numele tunnelului și domeniul):

```
# Exemplu: tunnelul existent se numește transitix-rai
cloudflared tunnel route dns transitix-rai documente-mistral.domeniul-tau.ro

# Verifică în Cloudflare Dashboard → DNS: CNAME documente-mistral → <tunnel-id>.cfargotunnel.com
```

7.3 Extinde `config.yml` cu al doilea ingress

Păstrează ruta veche; adaugă una nouă *deasupra* catch-all-ului `http_status:404`:

```
tunnel: transitix-rai
credentials-file: /root/.cloudflared/<TUNNEL-UUID>.json

ingress:
  # Instanța VECHE (neschimbată)
  - hostname: documente.domeniul-tau.ro
    service: http://127.0.0.1:3002
    originRequest:
      disableChunkedEncoding: false
      connectTimeout: 30s

  # Instanța NOUĂ MISTRAL (test)
  - hostname: documente-mistral.domeniul-tau.ro
    service: http://127.0.0.1:3003
    originRequest:
      disableChunkedEncoding: false
      connectTimeout: 30s

  - service: http_status:404
```

Ordinea contează: hostname-urile specifice înainte de `http_status:404`. Path-ul credentials-file rămâne al tunnelului tău existent (nu inventa alt UUID).

7.4 Actualizează `server/.env` pe instanța MISTRAL

```
CLIENT_ORIGIN=https://documente-mistral.domeniul-tau.ro
TRUST_PROXY=1
```

Apoi:

```
sudo systemctl restart transitix-mistral
sudo systemctl restart cloudflared
# sau: sudo cloudflared service restart

curl -sI https://documente-mistral.domeniul-tau.ro/api/health
curl -s https://documente-mistral.domeniul-tau.ro/api/health
# aștepți ocr: mistral
```

7.5 (Alternativă) Tunnel separat doar pentru MISTRAL

```
cloudflared tunnel create transitix-mistral-test
cloudflared tunnel route dns transitix-mistral-test documente-mistral.domeniul-tau.ro
# config nou + systemd unit separat, service: http://127.0.0.1:3003
```

Are sens doar dacă vrei izolare totală de config-ul vechi. Altfel, 7.2–7.4 e suficient.

7.6 Ce NU folosi pentru link-uri pe care le dai oamenilor

```
# Quick tunnel — hostname nou la fiecare restart, nu e pentru bookmark / Vercel
cloudflared tunnel --url http://127.0.0.1:3003
```

8. Vercel pe link-ul Cloudflare (frontend pe Vercel, API pe VM)

Context: clientul apelează `/api/…` și `/uploads/…` pe *același origin* (URL relative). Pe Vercel trebuie rewrite către hostname-ul Cloudflare al API-ului de pe VM. Alternativa mai simplă: **fără Vercel** — deschizi direct `https://documente-mistral…` (API-ul servește deja `dist/`).

Dacă tot ce vrei e „un link HTTPS de test", oprește-te după §7. Vercel are rost când vrei preview-uri / deploy UI separat de API.

8.1 Pregătire pe VM (API rămâne sursa de adevăr)

- Tunnel Cloudflare (§7) trebuie să răspundă pe `https://documente-mistral.domeniul-tau.ro/api/health`.
- În `server/.env`, `CLIENT_ORIGIN` trebuie să includă URL-ul Vercel:

```
# După ce știi domeniul Vercel (ex. transitix-mistral.vercel.app):
CLIENT_ORIGIN=https://transitix-mistral.vercel.app,https://documente-mistral.domeniul-tau.ro
TRUST_PROXY=1

sudo systemctl restart transitix-mistral
```

Dacă autentificarea acceptă o singură origină, folosește *doar* URL-ul Vercel în `CLIENT_ORIGIN` când testezi prin Vercel.

8.2 Fișier `vercel.json` în rădăcina repo

```
cat > vercel.json <<'EOF'
{
  "buildCommand": "npm run build:companion",
  "outputDirectory": "dist",
  "framework": null,
  "rewrites": [
    {
      "source": "/api/:path*",
      "destination": "https://documente-mistral.domeniul-tau.ro/api/:path*"
    },
    {
      "source": "/uploads/:path*",
      "destination": "https://documente-mistral.domeniul-tau.ro/uploads/:path*"
    },
    { "source": "/((?!api/|uploads/).*)", "destination": "/index.html" }
  ]
}
EOF
```

Înlocuiește hostname-ul cu domeniul tău real din §7. Fără rewrite-uri, login-ul pe Vercel lovește 404 pe `/api`.

8.3 Creează proiectul în Vercel (UI)

1. Intră pe **vercel.com** → **Add New** → **Project**
2. **Import** repo-ul `Georgiana-Ghimes/TransitiX`
3. **Root Directory:** rădăcina repo (nu `server/`)
4. **Framework Preset:** Other (build-ul e din `vercel.json`)
5. **Branch de producție:** `MISTRAL-experimenting-rai-documents`
6. Nu pune pe Vercel `MISTRAL_API_KEY` / `DATABASE_URL` — OCR și DB rămân pe VM
7. **Deploy**

8.4 CLI (alternativă)

```
npm i -g vercel
cd ~/apps/transitix-mistral
# vercel.json prezent (commit sau local)
vercel link
vercel --prod
```

8.5 După deploy — aliniază env pe VM

```
# URL dat de Vercel, ex. https://transitix-mistral-xxxx.vercel.app

CLIENT_ORIGIN=https://transitix-mistral-xxxx.vercel.app
TRUST_PROXY=1
sudo systemctl restart transitix-mistral

# Test: deschizi URL-ul Vercel → login → Network: /api/... = 200
# (proxied Vercel → Cloudflare → VM:3003)
```

8.6 Domeniu custom pe Vercel (opțional)

1. Vercel → Project → Settings → Domains → `app-mistral.domeniul-tau.ro`
2. DNS (în Cloudflare DNS): CNAME spre ce îți spune Vercel (de obicei `cname.vercel-dns.com`)
3. Actualizează `CLIENT_ORIGIN` pe VM

8.7 Limitări

- Upload / OCR foarte lung: timeout proxy Vercel poate tăia; pentru dosare grele folosește direct hostname-ul Cloudflare (§7).
- Push pe branch → Vercel rebuild UI; pe VM: `git pull` + migrate + restart API.
- Variantă curată: pe VM doar API; pe Vercel doar UI; tunnel Cloudflare expune `:3003`.

9. Checklist „ready for testing”

1. Docker DB healthy pe `:5436`
2. `GET /api/health` (local sau via Cloudflare) → `ocr: "mistral"`
3. HTTPS: `https://documente-mistral…/api/health` OK
4. (Dacă Vercel) login pe URL-ul Vercel; Network arată `/api` proxiat
5. Login admin → UI companion (Avize / Rapoarte / Setări)
6. Versiune UI: **v1.13.1 · build 73**
7. Upload PDF/poză → OCR → verificare câmpuri
8. Login șofer → Documente → poză / foaie 1–14
9. Fișiere pe VM în `server/uploads/`

10. Update după modificări pe branch

```
cd ~/apps/transitix-mistral
git pull origin MISTRAL-experimenting-rai-documents
npm ci
npm ci --prefix server
npm run db:migrate

# Dacă UI e servit de pe VM:
npm run build:companion
sudo systemctl restart transitix-mistral

# Dacă UI e pe Vercel: push pe branch pentru frontend;
# pe VM restartezi doar API-ul după migrate.
```

11. Ce să nu faci

- Nu folosi același `JWT_SECRET` / aceeași DB ca instanța veche.
- Nu rula `seed` pe o bază cu date reale de client.
- Nu deschide `:5436` / `:3003` spre internet dacă ai tunnel — lasă-le pe localhost.
- Nu pune `TRUST_PROXY=1` fără Cloudflare (sau alt proxy) în față.
- Nu da oamenilor un link de quick tunnel (`*.trycloudflare.com` efemer).
- Nu uita să actualizezi `CLIENT_ORIGIN` când schimbi între Cloudflare-only și Vercel.

Rezumat porturi & URL-uri

Ce	Unde
Postgres	Docker host `5436`
API (+ UI dacă build pe VM)	Node `127.0.0.1:3003`
HTTPS public (tunnel)	`https://documente-mistral.domeniul-tau.ro` → :3003
UI pe Vercel (opțional)	`https://….vercel.app` → rewrite `/api` & `/uploads` spre tunnel

---

Ghid deploy VM · branch MISTRAL-experimenting-rai-documents · companion v1.13.1 / build 73 · Cloudflare tunnel + Vercel · 1 octombrie 2026 · docs/ local only